8.2.4 Carefully Choose a Data Structure

Binary search on an array is cool, but arrays are not really known for their flexibility, if you remember what we discussed in chapter. If we would like to add another point between `W-3` and `B-2`, for instance, we would have to move all array entries points from `B-2` to `B-4`, and possibly reallocate the array, if it's a static array.

Luckily, we do know a data structure that is more flexible than arrays and that would allow us to perform a binary search efficiently: as its name suggests, a *binary search tree (BST)* is what we are looking for. In figure 8.5 we show a *balanced* binary search tree – we will need for the tree to be balanced to guarantee logarithmic running time on the most common operations.

For this example, we show a tree that contains both cities and warehouses; you can imagine that each city has a big warehouse or distribution center, and so our searches can just return the closest entry (either a city or a warehouse) to a customer (that is not in one of the cities in the tree).